

Memoria de Diseño: Arquitectura de Bits y Gestión de Memoria - Tetris

Realizado por: Juan José Cardona, Mateo Santa

Asignatura: Informática II - Universidad de Antioquia

1. Estrategia de Representación en Memoria

En lugar de utilizar una matriz de enteros, hemos diseñado un sistema de almacenamiento compacto utilizando el tipo de dato `unsigned char`. Esta decisión se fundamenta en la optimización de recursos: cada bloque del tablero se representa con un solo bit.

Configuración Dinámica:

El tablero se gestiona mediante un puntero doble (`unsigned char **pantalla`). Al inicializar el juego, se reserva memoria para n filas, donde cada fila es un arreglo de bytes. El número de bytes por fila se calcula como $\text{ancho} / 8$, garantizando que cada byte contenga exactamente 8 celdas de juego.

2. Análisis Algorítmico de la Manipulación de Bits

Para interactuar con el tablero, no accedemos a las celdas individuales como en una matriz común, sino que realiza operaciones booleanas sobre los bytes.

2.1. Localización de Celdas

Para cualquier coordenada (x, y) el acceso se divide en dos etapas:

1. **Índice de Byte:** Determinado por la parte entera de la división $x / 8$.
2. **Posición del Bit:** Calculado con el residuo $x \% 8$. Para que visualmente el bit más significativo esté a la izquierda, aplicamos la inversión $7 - \text{residuo}$.

2.2. Lógica de Modificación (Máscaras)

- **Activación:** Utilizamos una máscara con el operador **OR** (`|`). Esto permite "encender" un bit específico sin alterar el estado de los otros 7 bits contenidos en el mismo byte.
- **Consulta (Colisiones):** Mediante el operador **AND** (`&`), aislamos el bit en la posición deseada. Si el resultado de la operación es mayor a cero, el sistema interpreta una colisión física, disparando la lógica de detención de la pieza o el fin del juego.

3. Dinámica de Rotación y Validación de Movimientos

La rotación de las piezas no es solo mover las coordenadas. Se usa una validación previa:

1. **Predicción:** Antes de rotar, generamos coordenadas temporales aplicando una matriz de rotación de 90 grados.
2. **Auditoría de Bits:** Se recorren estas nuevas posiciones consultando el tablero (`leerBit`).
3. **Ejecución:** Solo si todos los bits de la nueva posición están "apagados" en el tablero es decir es un espacio vacío, se procede a actualizar la pieza.

4. Gestión del Ciclo de Vida de los Datos

Para dar cumplimiento de memoria dinámica, el programa sigue unos pasos estrictos de liberación:

- **Se mantiene:** La memoria permanece asignada durante toda la ejecución del bucle principal.
- **Destrucción Controlada:** Al detectar la señal de salida o el "Game Over", el software ejecuta un proceso de limpieza liberando primero cada arreglo de bytes (filas) y finalmente el puntero maestro, evitando así fugas de memoria.

5. Bitácora de Desafíos Técnicos

- **El problema de los bordes:** Un desafío importante fue mantener la integridad de los bordes laterales durante la eliminación de filas. Al desplazar los bytes de una fila a otra, los bordes tendían a desaparecer. La solución fue rediseñar la función de limpieza para que restaure las máscaras de los extremos en cada turno.
- **Sincronización de Dibujo:** Dado que el profesor solicitó re-impresión continua, ajustamos el flujo para que cada "cuadro" del juego se imprima como un bloque atómico, permitiendo una lectura clara del historial de movimientos en la terminal.
- **Probelmas con el acabado:** Fue un dolor de cabeza el acabado debido a que el código modificado daba bugs entonces teníamos que retornar al original además de algún que otro sabotaje del compañero porque no sabíamos la visión que tenía el otro entonces nos cambiábamos cosas .

6. Conclusiones

El desafío demuestra que la manipulación de bits no es solo una técnica de ahorro de espacio, sino una forma de entender la computación a bajo nivel. El sistema desarrollado es capaz de procesar un juego de lógica compleja utilizando una fracción mínima de la memoria.

Nos sentimos satisfechos con los resultados quizás a falta de tiempo no pudimos indagar de todas las formas que quisiéramos ya sea una implementación más limpia de cada factor o predicciones o hasta un registro del usuario (no es necesario, pero es un lindo detalle), dando así por finalizado nuestro Tetris el cual nos dio frustración de tantos bugs y cambios no advertidos, pero dejándonos el manejo de memoria dinámica mucho más claro